



Sistemas Embebidos I

Otoño 2026

**Práctica #2: Bitwise&GPIO software debouncing,
polling FSM**

María Fernanda Conde Calatayud

Andrea Ceja Lara

24/08/2026

Índice

<i>Práctica #2: Bitwise&GPIO: software debouncing, polling FSM</i>	2
Introducción	2
Descripción de la práctica	2
Objetivos	2
Marco teórico	3
Desarrollo	5
Materiales	5
Esquemático	5
Desarrollo	5
Resultados	8
Evidencias	8
Conclusiones	10

Práctica #2: Bitwise&GPIO: software debouncing, polling FSM

Introducción

Descripción de la práctica

En esta práctica se implementó un sistema antirrebote (debounce) para un botón físico conectado a una Raspberry Pi Pico, utilizando una máquina de estados finitos (FSM) para filtrar el ruido mecánico propio de los pulsadores y garantizar que cada pulsación real produzca exactamente un cambio de estado en un LED.

El botón, al presionarse o soltarse, genera múltiples transiciones eléctricas espurias (rebotes) durante unos pocos milisegundos antes de estabilizarse. Sin un filtro adecuado, esto provocaría que el sistema interprete una sola pulsación como varias, generando múltiples cambios no deseados en el LED.

Objetivos

- Diseñar el diagrama de estados de la FSM que modele el comportamiento del botón y el filtrado de rebotes en cada transición.
- Implementar el algoritmo de antirrebote utilizando las funciones de alto nivel del Pico SDK (`gpio_get()`, `gpio_put()`).
- Implementar la misma lógica accediendo directamente al periférico SIO.
- Armar el circuito físico correspondiente al sistema, tanto en simulación (Wokwi) como su representación física.
- Verificar el correcto funcionamiento del antirrebote mediante un protocolo de pruebas definido (pulsación simple, pulsación sostenida y pulsaciones rápidas consecutivas), validando que el conteo de cambios del LED sea el esperado en cada caso.
- Documentar el proceso mediante evidencia en video que demuestre el cumplimiento de cada prueba.

Marco teórico

Entradas digitales

Una entrada digital es un GPIO configurado para leer un nivel lógico: alto (1) o bajo (0). A diferencia de una salida, no impone voltaje; solo observa el que llega desde un botón, sensor digital o lógica externa (1). Un botón conectado a un GPIO normalmente se configura con una resistencia de pull-up (interna o externa), de modo que en reposo el pin lee 1 y, al presionar el botón, el pin se conecta a tierra y lee 0. Esta lectura se puede obtener de dos formas:

- **Mediante el SDK** (`gpio_get()` en el Pico SDK), que abstrae el acceso al hardware.
- **Mediante acceso directo a registros (SIO)**, leyendo el registro `GPIO_IN` del bloque *Single-cycle IO*, que expone el estado de todos los pines en un solo registro de 32 bits, uno por bit.

Esta segunda forma es la que conecta directamente con el uso de operaciones bitwise, ya que para saber el estado de un pin específico dentro de ese registro de 32 bits es necesario aislar el bit correspondiente (2).

Operaciones bitwise y máscaras

Las operaciones bitwise trabajan directamente sobre los bits individuales de una variable, en lugar de sobre su valor numérico completo. Las más relevantes para el manejo de GPIO son:

- **AND (&):** se usa para aislar uno o varios bits de un registro. Al aplicar una máscara (un valor donde solo el bit de interés está en 1 y todos los demás en 0), el resultado conserva únicamente ese bit.
- **OR (|):** se usa para activar (poner en 1) un bit sin alterar los demás, típicamente para encender un LED mediante el registro `GPIO_OUT_SET`.
- **AND con complemento (& ~mascara):** se usa para apagar (poner en 0) un bit específico sin tocar el resto.
- **XOR (^):** se usa para invertir (toggle) el estado de un bit; aplicar XOR dos veces con la misma máscara devuelve el bit a su valor original, lo cual es exactamente el comportamiento que necesita un LED que cambia de estado cada vez que se detecta una pulsación válida.
- **Desplazamiento (<<):** se usa para construir la máscara de un pin arbitrario, por ejemplo `1u << PIN_LED` genera un valor con un único bit en 1 en la posición del pin deseado (3).

En el contexto de esta práctica, las máscaras permiten trabajar a nivel de registro (SIO) sin depender de las funciones del SDK, leyendo y modificando bits puntuales del registro de entradas y salidas de forma eficiente .

Antirrebote (debounce)

Un botón mecánico no realiza una transición limpia entre sus dos estados. Al presionarlo o soltarlo, el contacto físico "rebota" durante unos pocos milisegundos, generando varias transiciones rápidas entre 0 y 1 antes de estabilizarse. Si el sistema lee el pin sin ningún tratamiento, interpretará ese rebote como varias pulsaciones distintas en lugar de una sola.

El antirrebote es la técnica que filtra ese ruido para que el sistema reaccione una sola vez por cada pulsación real (3). Puede implementarse de forma:

- **Por hardware**, con un capacitor y una resistencia (filtro RC) que suaviza la señal (1).
- **Por software**, ignorando cambios de estado durante una ventana de tiempo corta (típicamente 20–50 ms) después de detectar una transición, hasta confirmar que el nuevo estado es estable (1).

La implementación por software es la más común en sistemas embebidos porque no requiere componentes adicionales, y es la que se desarrolla en esta práctica mediante una máquina de estados.

Máquina de estados finitos (FSM)

Una FSM es un modelo de comportamiento compuesto por un número finito de estados, transiciones entre esos estados y eventos que disparan dichas transiciones. En cada instante el sistema se encuentra en un único estado, y solo cambia de estado cuando ocurre un evento definido para esa transición (4).

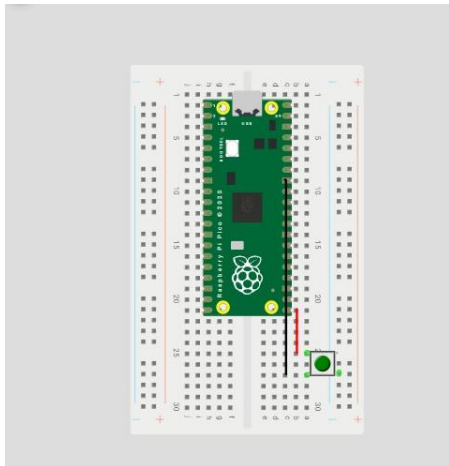
Aplicar una FSM al antirrebote permite separar claramente la lógica de "cuándo confirmar una pulsación" de la lógica de "qué hacer cuando la pulsación es válida" (en este caso, alternar el LED). En lugar de usar únicamente retardos (delay) que bloquean el programa, la FSM evalúa en cada ciclo el estado actual y la entrada del botón, y decide si permanece en el mismo estado o transiciona a otro, lo que permite que el sistema siga respondiendo (por ejemplo, atendiendo otros periféricos) mientras espera a que el rebote se estabilice (3).

Desarrollo

Materiales

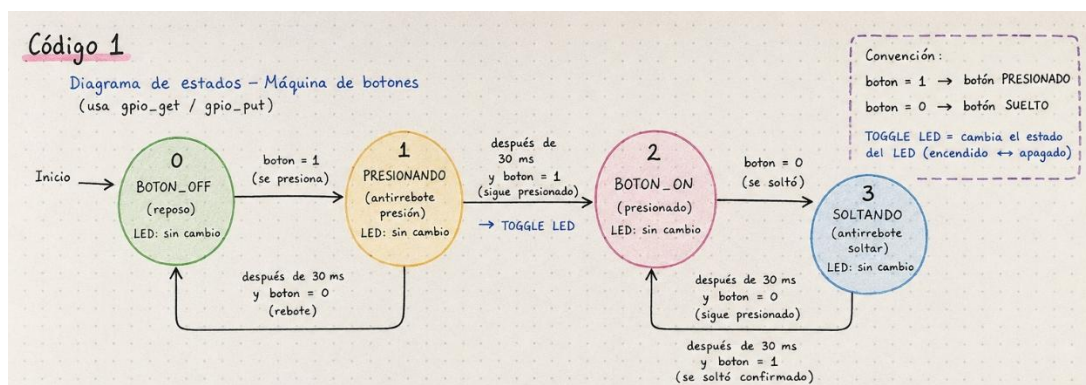
- Protoboard
- Raspberry Pi Pico
- Botón
- Jumpers
- Simulador wowki

Esquemático



Desarrollo

Diagrama de estados (código 1)



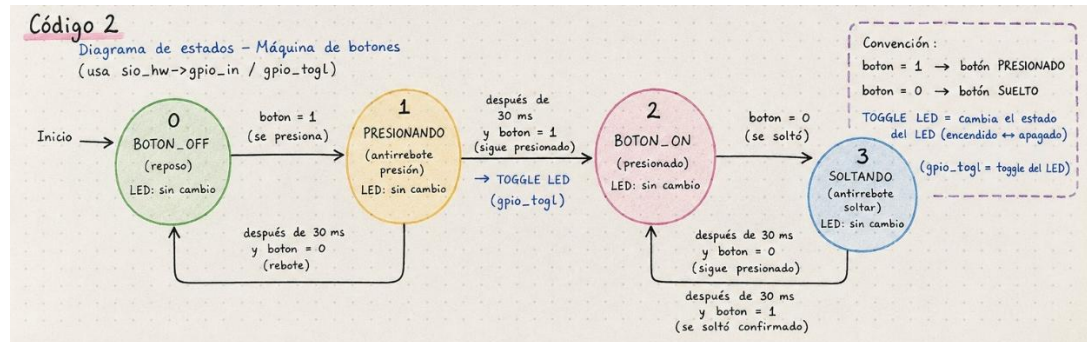
Código 1

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #define BTN_PIN 16 //define el pin del gpio al boton
5  #define DEBOUNCE_DELAY_MS 30 //tiempo de espera para el antirrebote
6
7  typedef enum {
8      BOTON_OFF, //estado 0: boton en reposo
9      PRESIONANDO, //estado 1: espera la confirmacion del antirrebote
10     BOTON_ON, //estado 2: se confirma el pulso, se espera que se suelte
11     SOLTANDO //estado 3: se detecta que se suelta
12 } EstadoFSM;
13
14 int main() {
15     stdio_init_all(); //inicia la comunicacion serial
16
17     const uint LED_PIN = PICO_DEFAULT_LED_PIN; //define el led de la raspberry
18     gpio_init(LED_PIN); //configura la funcion para el led
19     gpio_set_dir(LED_PIN, GPIO_OUT); //pin del led como salida
20     gpio_put(LED_PIN, 0); //led iniciando en apagado
21
22     gpio_init(BTN_PIN); //inicia el boton
23     gpio_set_dir(BTN_PIN, GPIO_IN); //boton como entrada
24     gpio_pull_up(BTN_PIN); //reposo = 1 y presionado = 0
25
26     EstadoFSM estado_actual = BOTON_OFF; //maquina de estados en reposo
27     uint32_t tiempo_inicio = 0; //guarda el tiempo cuando inicia la maquina de estado
28     bool led_state = false; //guardar el estado del led
29
30     while (true) {
31         bool btn_lectura = gpio_get(BTN_PIN); //lee el boton
32         uint32_t tiempo_actual = to_ms_since_boot(get_absolute_time()); //tiempo desde que empezo la operacion
33
34         switch (estado_actual) {
35             case BOTON_OFF:
36                 if (btn_lectura == 0) { //boton presionado conectado a tierra
37                     tiempo_inicio = tiempo_actual; //guarda el tiempo inicial
38                     estado_actual = PRESIONANDO; //avanza al estado de antirrebote
39                 }
40                 break;
41
42             case PRESIONANDO:
43                 if (btn_lectura == 1) { //si detecta 1 antes de tiempo fue ruido
44                     estado_actual = BOTON_OFF; //cancela y regresa al inicio
45                 } else if ((tiempo_actual - tiempo_inicio) >= DEBOUNCE_DELAY_MS) { //si ya pasaron los 30 ms
46                     led_state = !led_state; //cambia el valor del led
47                     gpio_put(LED_PIN, led_state); //actualiza el valor del led
48                     estado_actual = BOTON_ON; //pasa al estado para que se suelte
49                 }
50                 break;
51
52             case BOTON_ON:
53                 if (btn_lectura == 1) { //detecta 1, se empieza a soltar el boton
54                     tiempo_inicio = tiempo_actual; //tiempo cuando se empezo a soltar el boton
55                     estado_actual = SOLTANDO; //pasa al estado de antirrebote al soltar
56                 }
57                 break;
58
59             case SOLTANDO:
60                 if (btn_lectura == 0) { //si vuelve a 0 fue rebote mecanico
61                     estado_actual = BOTON_ON; //regresa al estado presionado
62                 } else if ((tiempo_actual - tiempo_inicio) >= DEBOUNCE_DELAY_MS) { //verifica el tiempo de estar suelto
63                     estado_actual = BOTON_OFF; //regresa al reposo inicial
64                 }
65                 break;
66         }
67         sleep_ms(1); //pausa para no saturar el bus

```

Diagrama de estados (código 2)



Código 2

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "hardware/structs/sio.h"
4
5  #define BTN_PIN 16 //define el pin del gpio al boton
6  #define LED_PIN PICO_DEFAULT_LED_PIN //define el pin del led de la raspberry
7  #define DEBOUNCE_DELAY_MS 30 //tiempo de espera para el antirrebote
8
9  const uint32_t BTN_MASK = (1u << BTN_PIN); //mascara de bits para el pin del boton
10 const uint32_t LED_MASK = (1u << LED_PIN); //mascara de bits para el pin del led
11
12 typedef enum {
13     BOTON_OFF, //estado 0: boton en reposo
14     PRESIONANDO, //estado 1: espera la confirmacion del antirrebote
15     BOTON_ON, //estado 2: se confirma el pulso, se espera que se suelte
16     SOLTANDO //estado 3: se detecta que se suelta
17 } EstadoFSM;
18
19 int main() {
20     stdio_init_all(); //inicia la comunicacion serial
21
22     gpio_init(LED_PIN); //configura la funcion para el led
23     gpio_init(BTN_PIN); //configura la funcion para el boton
24
25     sio_hw->gpio_oe_set = LED_MASK; //pin del led como salida mediante registro sio
26     sio_hw->gpio_oe_clr = BTN_MASK; //pin del boton como entrada mediante registro sio
27     gpio_pull_up(BTN_PIN); //reposo = 1 y presionado = 0
28
29     sio_hw->gpio_clr = LED_MASK; //led iniciando en apagado mediante registro sio
30
31     EstadoFSM estado_actual = BOTON_OFF; //maquina de estados en reposo
32     uint32_t tiempo_inicio = 0; //guarda el tiempo cuando inicia la maquina de estado
33     bool led_state = false; //guardar el estado del led
34
35     while (true) {
36         bool btn_presionado = !(sio_hw->gpio_in & BTN_MASK); //lee el registro y da 1 si esta presionado a tierra
  
```

```

37     uint32_t tiempo_actual = to_ms_since_boot(get_absolute_time()); //tiempo desde que empezo la operacion
38
39     switch (estado_actual) {
40     case BOTON_OFF:
41         if (btn_presionado) { //boton presionado conectado a tierra
42             tiempo_inicio = tiempo_actual; //guarda el tiempo inicial
43             estado_actual = PRESIONANDO; //avanza al estado de antirrebote
44         }
45         break;
46
47     case PRESIONANDO:
48         if (!btn_presionado) { //si se suelta antes de tiempo fue ruido
49             estado_actual = BOTON_OFF; //cancela y regresa al inicio
50         } else if ((tiempo_actual - tiempo_inicio) >= DEBOUNCE_DELAY_MS) { //si ya pasaron los 30 ms
51             sio_hw->gpio_togl = LED_MASK; //invierte el estado del led directamente en sio
52             estado_actual = BOTON_ON; //pasa al estado para que se suelte
53         }
54         break;
55
56     case BOTON_ON:
57         if (!btn_presionado) { //detecta que se empieza a soltar el boton
58             tiempo_inicio = tiempo_actual; //tiempo cuando se empezo a soltar el boton
59             estado_actual = SOLTANDO; //pasa al estado de antirrebote al soltar
60         }
61         break;
62
63     case SOLTANDO:
64         if (btn_presionado) { //si vuelve a presionarse fue rebote mecanico
65             estado_actual = BOTON_ON; //regresa al estado presionado
66         } else if ((tiempo_actual - tiempo_inicio) >= DEBOUNCE_DELAY_MS) { //verifica el tiempo de estar suelto
67             estado_actual = BOTON_OFF; //regresa al reposo inicial
68         }
69         break;
70     }
71
72     sleep_ms(1); //pausa para no saturar el bus
73 }
74

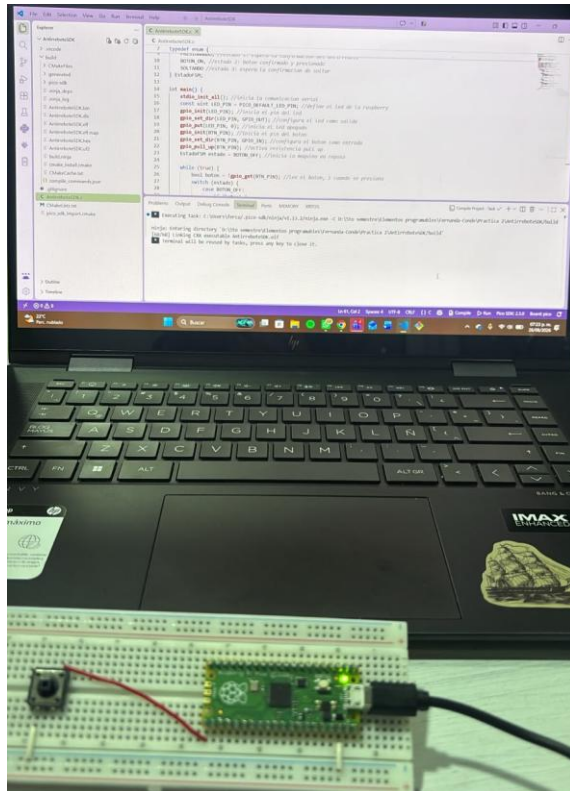
```

Resultados

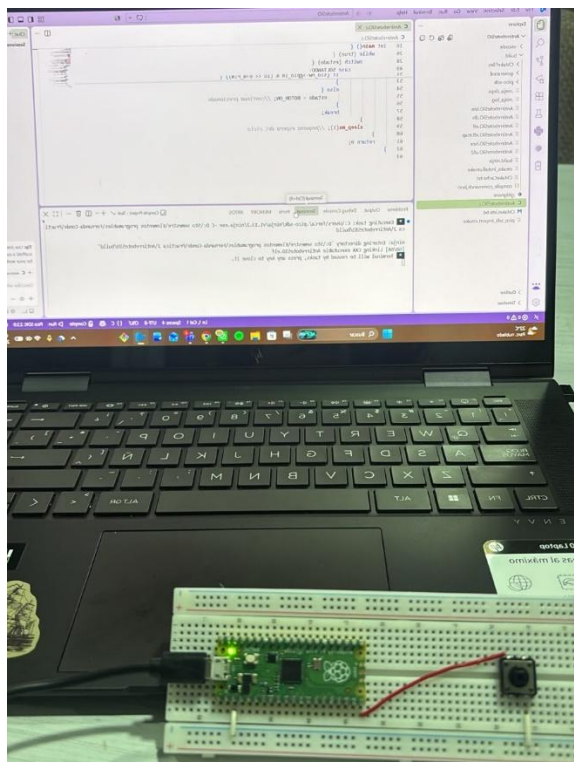
Prueba	Resultado esperado	Resultado obtenido
Presionar una vez el botón	El LED cambia exactamente 1 vez	Al presionar una vez el botón, el LED cambia 1 vez
Mantener el botón presionado 3s	El LED se mantiene estable por 3s	Al mantener el botón presionado por 3s, el led se mantiene prendido por 3s
Presionar el botón 5 veces seguidas	Se cuentan 5 cambios, ni mas ni menos	Al presionar el botón 5 veces seguidas, prende y apaga 5 veces

Evidencias

SDK



SIO



Conclusiones

La FSM de antirrebote funcionó correctamente en las tres pruebas, ya que permitió distinguir una pulsación real de los rebotes del botón. Además, comparar el código SDK con el SIO ayudó a entender mejor cómo funcionan los registros, las máscaras y las operaciones bitwise. En general, la práctica permitió comprender la importancia de usar una máquina de estados para controlar entradas reales de forma más precisa.

Bibliografías

- [1] «Bare-Metall/O&Timing Bitwise&GPIO: Softwaredebouncing, PollingFSM», agosto de 2026.
- [2] E. Rios, «ENTRADAS DIGITALES.», *STEAM Academy*, 23 de agosto de 2021. <https://www.steamacademy.com.co/post/entradas-digitales>
- [3] OpenAI. (2026). *ChatGPT* [Modelo de lenguaje]. <https://chatgpt.com>
- [4] Spiceworks, *Finite State Machine Meaning, Working, and Examples* | *Spiceworks*. 2025. [En línea]. Disponible en: <https://www.spiceworks.com/soft-tech/what-is-fsm/>